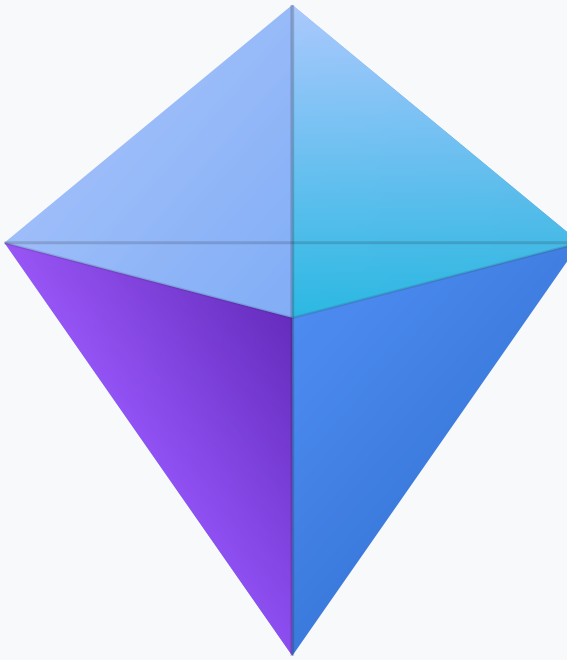
Навыки агентов

Авторы: Танви Сингхал, Габриэла Эрнандес Лариос,
Дебаншу Дус, Лави Нигам и Смита Колан



Google

Благодарности

Кураторы и редакторы
Шубхам Сабу

Дизайнер
Майкл Лэннинг

Содержание

1. **Введение** 5

2. **Что такое навык агента (и как собрать свой первый)** 6

- Анатомия навыка и прогрессивное раскрытие 6
- Путь A: перевод того, что вы уже знаете 7
- Путь B: кристаллизация того, что агент только что сделал 8
- Пробуем и устанавливаем навыки сегодня 9
- Чем навык отличается от MCP и AGENTS.md? 10

3. **Почему навыки агентов так быстро стали популярными**? 11

- Навыки для чат-ботов, кодинг-агентов или мультиагентных корпоративных сценариев? 12

4. **Оценка навыков** 14

- Набор инструментов оценки 15
- Триггер — первый шлюз 17
- Качество вывода и траектория инструментов 18
- Система против навыка: иллюзия оценки 18
- Бюджет токенов: изоляция — ловушка 19
- Что означает «полнота оценки» 21

5. От прототипа к продакшену 22
Что на самом деле внутри агентного рантайма 23
Почему навыки — единица улучшения 23
Сбой, ломающий демо: переполнение контекста 24
Что это значит для бюджета токенов 25

6. О мета-навыках и самоулучшающихся навыках 27
Где это ломается 28
Куда это движется 29

7. Композиция и упаковка навыков 30

8. Как выбрать среди сотен существующих навыков 33

9. Заключение 34

Приложение A — Практическая шпаргалка 35

Приложение B — Кейс: экспертиза домена как код (вертикальные навыки в ретейле) 40

Сноски 45

1. Введение

Навыки агентов (Agent Skills) — это способ наделить вашего агента знаниями и контекстом компании. Навык агента — это папка, которая содержит файл `SKILL.md`, а также директории `scripts/`, `references/` и `assets/`. Раздел 2 подробно разбирает анатомию навыка.

Навыки агентов стремительно становятся стандартом кроссплатформенной переносимости. Но чем объясняется такая скорость принятия? Мы считаем, что навыки агентов решают четыре ключевые точки трения в разработке ИИ-агентов:

1. **Слишком много инструкций — хуже результат.** Если свалить в один системный промпт все инструкции, какие только можно придумать, качество работы большой языковой модели (LLM) неизбежно деградирует. Эта проблема известна как «гниение контекста» (context rot). Навыки решают её, загружаясь исключительно по требованию. Раздел 5 разбирает исследования, стоящие за этим утверждением.
2. **Знать «как», а не только «что».** У LLM уже есть работающие аналоги для запоминания событий (эпизодическая память) и фактов (семантическая память). Чего им недоставало — так это способа запоминать, как выполнять действия пошагово; это называется процедурной памятью. Навыки агентов можно считать первым достоверным примитивом процедурной памяти для LLM-агентов.
3. **Перегрузка мультиагентностью.** Экосистему наводнили сложные мультиагентные системы, которые печально известны трудностью построения и сопровождения. Хотя для отдельных задач они по-прежнему необходимы, навыки позволяют одному универсальному агенту легко разворачиваться в множество специализированных ролей. Раздел 3 разрабатывает этот аргумент вглубь, на рабочем примере.
4. **Переносимость.** Папка с markdown-файлом — поразительно лёгкий примитив. Любой агент с доступом к файловой системе сможет им воспользоваться, а значит, навыки идеально переносимы в мультивендорном ИИ-ландшафте.

В этом отчёте мы обращаемся к двум типам читателей: **Строителям** (тем, кто использует навыки) и **Разработчикам** (тем, кто создаёт, версионирует и управляет ими). Мы мягко проведём вас через то, что такое навык и как им пользоваться (разделы 2–3), а затем погрузимся в сложные темы для разработчиков: оценку, готовность к продакшену, мета-навыки и композицию (разделы 4–8).

(Торопитесь? В Приложении A — печатная операционная шпаргалка, в Приложении B — кейс из ретейла).

2. Что такое навык агента (и как собрать свой первый)

Навык агента — это примитив, который даёт универсальному агенту специализированную компетенцию по требованию. Да, навык может быть настолько простым, насколько прост один markdown-файл, но не обязан на этом останавливаться — парадигма, стоящая за ним, вполне инновационна.

Сегодня навыки агентов возникают через два разных паттерна.

Первый путь — от экспертов предметной области, у которых институциональные знания уже где-то записаны. Представьте специалиста по комплаенсу с тридцатистраничным регламентом или HR-менеджера с гайдами по онбордингу новичков. Ни тому, ни другому не нужно учиться программировать, чтобы написать навык и начать им пользоваться: контент у них уже есть, осталось лишь перевести его в формат, который агент сможет использовать с умом.

Второй путь — разработчики, оборачивающие агентные или программные рабочие процессы в навыки. Если агент успешно выполнил нетривиальную повторяющуюся задачу, не хочется, чтобы в следующий раз он заново изобретал процесс с нуля. Вместо этого вы хотите, чтобы агент создал навык по итогам этого успешного запуска. Коротко: мы наблюдаем формирующийся паттерн — всё, что является хорошим переиспользуемым workflow, становится навыком; и писать его не обязательно — это делает агент, а вы рецензируете. Это уже территория мета-навыков, которую мы здесь мягко вводим, прежде чем погрузиться в неё в разделе 6.

Обе группы производят один и тот же артефакт — папку навыка с ядром-примитивом `SKILL.md`, — но пути к нему разные.

Анатомия навыка и прогрессивное раскрытие

Прежде чем рассматривать разные пути, разберём анатомию навыка. Каждый навык живёт в собственной директории и обязан содержать `SKILL.md`. Чтобы увидеть полную каноническую структуру, определённую открытым стандартом agentskills.io, посмотрим на практический пример.

Ниже — иллюстративная папка навыка для ежедневной подготовки кафе. (Помните: единственный обязательный файл — `SKILL.md`. Всё остальное опционально):

```
cafe-preparation/
├── SKILL.md                       # Обязательно: метаданные + инструкции
├── scripts/                       # Опционально: исполняемый код (любой язык)
│   ├── calc_quantities.py         # Расчёт латте, круассанов и т.д.
│   └── convert_to_ingredients.py # Перевод напитков и выпечки в ингредиенты
├── references/                    # Опционально: дополнительный контекст
│   ├── menu_and_recipes.md        # Матча-латте: 3 г порошка, 200 мл...
│   └── minimums.md                # Всегда готовить минимум 40 латте...
└── assets/                        # Опционально: шаблоны, конфиги, схемы
    ├── prep_sheet_template.md     # Итоговая раскладка для кухонного персонала
    └── shopping_list_template.md # Раскладка для заказа у поставщиков
```

Фрагмент 1: дерево директорий навыка — стандартная раскладка и дизайн прогрессивного раскрытия для навыка подготовки кафе.

Инновационная часть — **прогрессивное раскрытие**. Навыки загружаются на трёх уровнях:

1. **Метаданные** (имя + описание) всегда находятся в контексте агента.
2. **Тело SKILL.md** загружается, только когда навык срабатывает.
3. **Вложенные ресурсы** загружаются строго по мере необходимости (а скрипты выполняются, вообще не загрязняя окно токенов).

Это значит: у вас может быть сто установленных навыков, но на каждом ходу вы платите крошечную стоимость токенов только за их метаданные. Перейдём к практике — как собрать первый навык.

Путь A: перевод того, что вы уже знаете

Минимальный шаблон `SKILL.md` лежит в Приложении A — его можно скопировать целиком. Прежде всего сосредоточьтесь на YAML-frontmatter, потому что именно он является триггером активации:

```
---
name: cafe-preparation
description: |
  Рассчитывает ежедневные потребности в ингредиентах и формирует
  подготовительные листы для кафе.
  Используй, когда пользователь просит оценить дневные объёмы,
  перевести напитки в ингредиенты или собрать список закупок.
  НЕ используй для составления сменного графика сотрудников
  или бухгалтерского учёта.
---
```

Фрагмент 2: YAML-frontmatter для навыка подготовки кафе.

С самого начала стоит правильно сделать две вещи: **имя** и **поле description**.

Именование. Будьте очевидны и скучны: `snake_case` для директорий (например, `bigquery_ingestion`), `kebab-case` для имён навыков (например, `pdf-processing`), предпочитайте герундий: `managing-databases`. Избегайте общих имён вроде `utils` или `tools` и не используйте внутренний жаргон.

Описание. Это ваш алгоритм маршрутизации. Единственное, что видит модель, решая, загружать ли навык. Опишите, что навык делает, вынесите триггерные ключевые слова в начало, будьте настойчивы, если модель срабатывает неохотно, и явно укажите, для чего навык *не* предназначен.

Когда `SKILL.md` набросан, вы достраиваете остальную папку. Здесь прогрессивное раскрытие начинает окупаться: всё, что не обязано находиться в теле `SKILL.md`, уходит в другое место:

- **Скрипты.** Детерминированная работа (парсинг выгрузок, расчёты, форматирование) живёт в `scripts/`. Модель решает, что делать; скрипт делает тяжёлую работу.
- **Справочники.** Знания, релевантные только пока навык работает (принципы предметной области, определения, обработка крайних случаев), живут в `references/` и загружаются по требованию.
- **Ассеты.** Шаблоны и схемы — в `assets/`.

Правило большого пальца: если `SKILL.md` начинает разрастаться, следующий абзац, скорее всего, должен жить в `references/`, а не в теле.

(Торопитесь? В Приложении A — печатная шпаргалка с курированными «делать/не делать» для первых шагов).

Путь B: кристаллизация того, что агент только что сделал

Второй путь начинается с другого конца. Вместо перевода того, что у вас уже есть, вы кристаллизуете то, что агент только что сделал.

Агент успешно выполнил задачу. Вы заметили, что процесс переиспользуем. Вы хотите, чтобы следующее подобное задание выиграло от только что накопленного опыта.

Это территория мета-навыков: навыков, чья задача — захватывать или улучшать другие навыки. Инструменты вроде `skill-creator` от Anthropic, Hermes Agent от Nous Research и паттерн self-improving-agent-skills из awesome-llm-apps поддерживают такой процесс. Агент наблюдает за успешной траекторией и предлагает черновик `SKILL.md`. Вы рецензируете, а не пишете.

Действует та же планка качества. Проверенный, итерированный черновик агента может быть отличным. Неотрецензированный черновик часто хуже, чем отсутствие навыка вовсе. К этой теме мы вернёмся с куда большей глубиной в разделе 6, посвящённом мета-навыкам.

Пробуем и устанавливаем навыки сегодня

Когда папка готова (написали её вы или агент), пора пробовать. Положите её в правильное для вашего инструмента место, перезапустите агента и проверьте естественным запросом. Следите за трассировкой, чтобы убедиться, что навык действительно сработал. Затем попробуйте запрос, на который он срабатывать *не должен*, и убедитесь, что навык молчит.

Но где именно «правильное место»?

Здесь всё становится тоньше — и это один из минусов взрывной популярности навыков. Каждый агент и кодинг-инструмент сошёлся на формате, но каждый ищет навыки в слегка разном месте. В целом сегодня есть три парадигмы взаимодействия с навыками и их установки:

1. **Файловый способ (кодинг-агенты и CLI).** Для локальных окружений паттерн файловый: вы кладёте папку навыка в определённую скрытую директорию, и агент её подхватывает. Хотя вокруг общей папки `.agents/skills/` в корне проекта складывается крайне желанная кроссинструментальная конвенция, многие инструменты всё ещё используют собственные пути. (Совет: если вы переключаетесь между несколькими CLI-инструментами и IDE, менеджеры вроде skillport или openskills автоматически создадут симлинки вашей центральной библиотеки навыков в ожидаемые места каждого инструмента).
2. **UI-установка (веб- и корпоративные воркспейсы).** Если вы работаете в веб-платформах совместной работы или потребительских чат-ботах, вы редко касаетесь терминала или скрытых папок. Такие платформы позволяют устанавливать, загружать и управлять навыками через визуальный UI-реестр в несколько кликов, беря на себя маршрутизацию для всей команды.
3. **Программный путь (кастомные фреймворки).** Если вы строите собственных специализированных агентов с нуля (например, на Google Agent Development Kit), вы загружаете навыки программно: указываете коду путь к папке — скажем, регистрируете её через класс `SkillToolset`, который автоматически генерирует под капотом необходимые инструменты маршрутизации `load_skill` для модели.

Паттерн везде одинаков: положите папку навыка в нужную директорию, перезапустите агента — и он подхватит навык. Меняется только «нужная директория» в зависимости от инструмента.

Совет: прежде чем предполагать, сверьтесь с документацией вашего конкретного кодинг-агента или чат-бота. Формат общий, а путь установки, правила активации и детали каждого инструмента (белые списки разрешённых инструментов, гейты безопасности, бандлинг плагинов) — нет.

Чем навык отличается от MCP и AGENTS.md?

Чтобы установить архитектурное место навыков агентов, полезно сопоставить эти примитивы.

Навык против MCP. Они не конкурируют — они компонуются. Model Context Protocol отвечает за *досягаемость*: MCP-сервер подключает агента к внешней системе (Drive, Salesforce, BigQuery или внутреннему API). Навык отвечает за *ноу-хау*: он учит агента думать о конкретном виде работы. Когда навыку нужны данные, он велит агенту вызвать инструмент — как правило, предоставляемый MCP-сервером.

Навык против AGENTS.md. С одной стороны, `AGENTS.md` всегда загружен внутри проекта; навыки загружаются по требованию. Самые чистые настройки используют оба. Держите `AGENTS.md` компактным (конвенции проекта, стек, команды сборки и т.д.) и, если нужно, используйте его и как роутер в библиотеку навыков: краткий каталог внизу подскажет агенту, что доступно.

3. Почему навыки агентов так быстро стали популярными?

Представьте начало 2025 года. Вас просят построить систему, которая возьмёт на себя повторяющуюся офисную работу: генерировать слайд-деки из брифов в фирменном стиле компании, разбирать структурированные PDF-счета, составлять HR-гайды по онбордингу, еженедельно сводить комплаенс-отчёты — и добивать длинный хвост подобных задач, который неизбежно растёт по мере того, как команда находит, что ещё автоматизировать.

Скорее всего, вы бы по умолчанию выбрали мультиагентную архитектуру: агент-роутер сверху раздаёт задачи горстке специализированных субагентов снизу. А затем провели бы мучительные часы в CI/CD-пайплайнах, над логикой оркестрации и над тем, чтобы деплой нового HR-субагента не сломал субагента для счетов.

С появлением навыков агентов этот процесс радикально упрощается. Именно это трение и породило формат навыков: первую реализацию создала Anthropic — первые навыки для чтения PDF и создания слайдов, — и это гораздо более лёгкий способ достичь того же результата. Вместо роутера, раздающего задачи субагентам, — один агент с библиотекой навыков. Навыки умеют запускать команды, вызывать MCP-серверы и включать в себя Python-скрипты. Агент сам решает, что и когда загрузить. Вы поддерживаете навыки, а не агентов — и операционная поверхность может сжаться.

> Будем предельно ясны: навыки агентов **не** убивают мультиагентные архитектуры.

Мультиагентность остаётся абсолютно верным ответом, когда есть настоящий параллелизм, реальные границы возможностей (разные доступы, разные режимы безопасности, разные внешние системы), иерархическая декомпозиция с действительно различающимися слоями абстракции, состязательные схемы со сдержками и противовесами, межсубагентная коммуникация или гетерогенные модели. Список не исчерпывающий.

Что сделали навыки — так это ввели недостающий архитектурный примитив. Многие системы, которые строились мультиагентными *по умолчанию*, теперь можно элегантно упростить до схемы «один агент с навыками» *по дизайну*.

Навыки для чат-ботов, кодинг-агентов или мультиагентных корпоративных сценариев?

Первые публично замеченные навыки имели форму ИИ-чат-ботов. Нарратив кодинг-агентов пришёл в течение дней или недель. А как только пришёл — навыки взорвались. Они попали прямо в лихорадку vibe-coding и оказались тем форматом, которого разработчики ждали.

В некоторых мультиагентных архитектурах каждый субагент уже по определению специалист. Исследовательскому агенту может не понадобиться исследовательский навык. Навыки полезнее в сценариях, где один универсальный агент по дизайну способен становиться специалистом в разных областях. Впрочем, бывают случаи, где мультиагентность и навыки хорошо компонуются — например, когда каждому специализированному агенту нужна своя ограниченная библиотека навыков.

Теперь рассмотрим логистическую компанию со 100 вариантами процессов: в зависимости от типа продукта, инструментов, ограничений маршрутов, SLA клиентов, регуляторных зон и так далее. Как решить задачу элегантно и легковесно?

- **Один агент, одно гигантское окно контекста.** Вызывает немедленное гниение контекста и непомерные расходы токенов.
- **RAG по регламентам.** Вероятно, правильный ответ два года назад. Но теперь вы тянете векторную базу, эмбеддинг-модель и стратегию чанкинга, чьё качество не имеет ничего общего с реальными процедурами.
- **Мультиагентность: субагент на каждый процесс.** 100 субагентов, у каждого свой системный промпт. Операционный кошмар: 100 деплоев, 100 поверхностей оценки, сложные слои маршрутизации.
- **Один агент, 100 навыков.** Это идеально ложится в формат навыков — множество вариантов одной и той же работы. Прогрессивное раскрытие означает, что 100 навыков стоят ~100 × 50 токенов = ~5 000 токенов постоянно загруженных метаданных. Логистические запросы несут сильные сигналы активации: SKU, пункт отправления, вес, флаг опасного груза, SLA — поэтому описания навыков получаются острыми, а выбор надёжным. Процедуры живут в системе контроля версий. Добавить 101-й вариант — это новая папка, а не новый деплой. Легче поддерживать и масштабировать.

Однако самое важное — всегда иметь строгий процесс оценки и сравнивать разные показатели, прежде чем принимать окончательное решение (раздел 4 описывает, как такая оценочная работа выглядит на практике).

Мысленная заметка: принятие идёт по пути наименьшего сопротивления. Тот, кто умеет писать документацию, сумеет написать и навык. Это снижает порог входа, и латентное процедурное

знание, пылящееся в вики, регламентах и головах инженеров, наконец обретает структурированное русло.

4. Оценка навыков

Итак, у вас есть первый навык — а может, и небольшая библиотека. Тут же встаёт вопрос: как понять, что они действительно работают? Ниже — как навыки отказывают, как их тестировать и какие четыре условия должен пройти каждый навык, прежде чем заслужить место в вашей библиотеке.

> Навык агента без теста — это надежда, а не способность.

Когда исследователи недавно прогнали 84 реальные агентные задачи через SkillsBench (2025), выяснилось: 19% задач выполнялись *хуже* с навыком, чем без него. Плохо спроектированные навыки были не просто нейтральным шумом — они активно ухудшали способности. К счастью, эти отказы предсказуемы и распадаются на четыре чётких режима:

1. **Отказ триггера:** срабатывает не тот навык — или нужный навык не срабатывает вовсе.
2. **Отказ исполнения:** навык срабатывает верно, но выдаёт неверный результат или ошибочные вызовы инструментов.
3. **Отказ бюджета токенов:** массивное тело навыка забивает окно контекста и ухудшает производительность на несвязанных ходах.
4. **Регрессия:** новый навык пересекается с существующим, ломая прежде работавшую маршрутизацию.

Отказы триггера всплывают в журналах маршрутизации; отказы исполнения — в качестве вывода; отказы бюджета токенов — под реалистичной контекстной нагрузкой; регрессии — только когда вся библиотека работает одновременно.

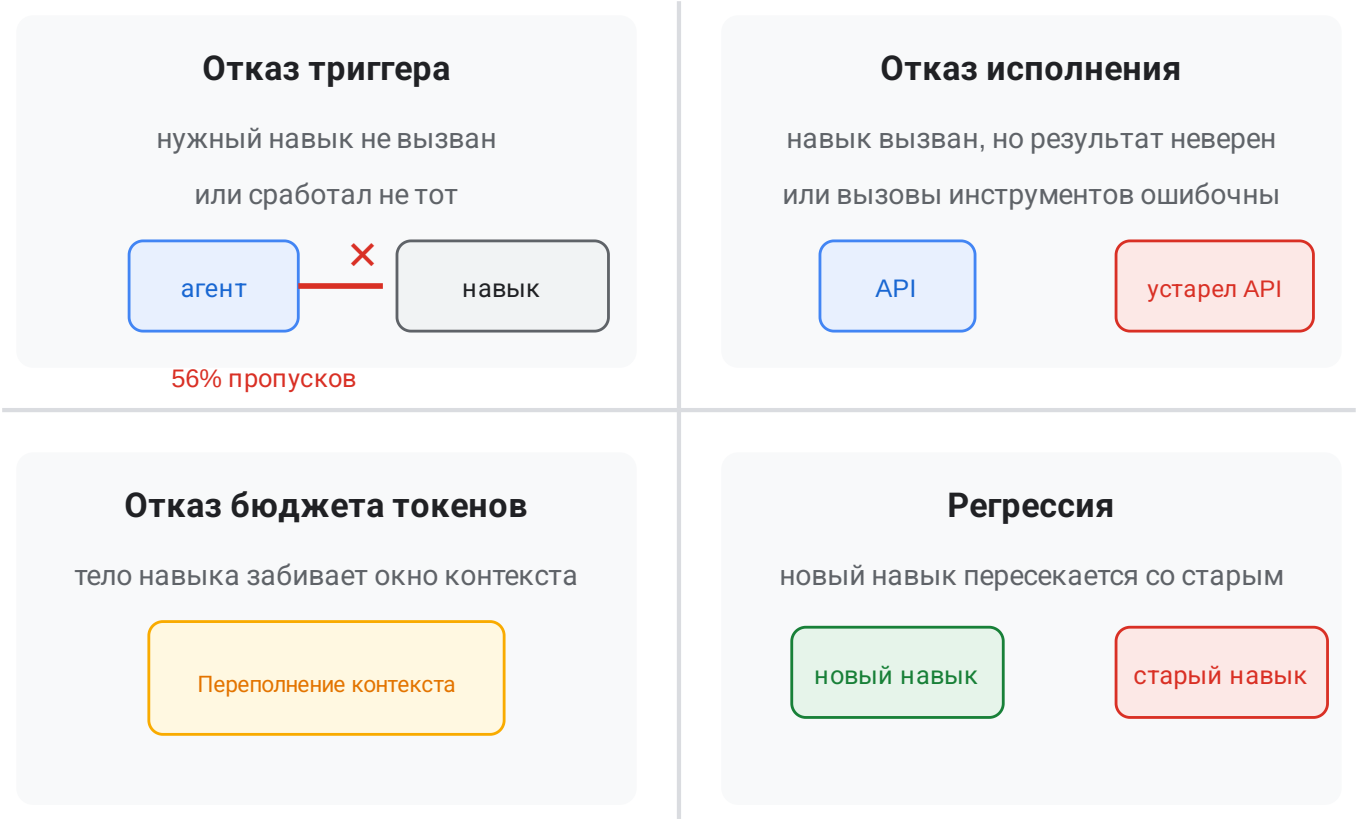


Рис. 1. Режимы отказов ветвятся: сбои триггера и исполнения происходят на уровне одного хода, а перерасход токенов и регрессии проявляются лишь при взаимодействии множества навыков под тяжёлой прод-нагрузкой.

Набор инструментов оценки

Пять взаимодополняющих паттернов тестирования покрывают всю поверхность отказов.

Паттерн	Описание	Пример	Режим отказа	Когда нужен
Оценка как юнит-тест	Тестовый файл навыка, запускаемый в CI на каждое изменение	Три JSON-кейса выполняются через agenteval на каждый push; падающий тест блокирует слияние	Все	Каждый навык, каждое изменение
Golden-датасет	Курируемые версионируемые пары (вход, ожидаемый вывод), хранящиеся вместе с навыком	30 репрезентативных запросов с ожидаемыми вызовами инструментов и форматами в директории навыка	Исполнение, триггер	Уровень draft и выше
LLM-судья	Модель-сверстник оценивает вывод по рубрике в масштабе	Скоринг по трём измерениям рубрики, дважды, с перестановкой позиций для устранения предвзятости порядка	Исполнение	Read-only и draft
Состязательный / red-team	Систематический поиск, спроектированный вскрывать режимы отказов	Один перефразированный и один негативный граничный кейс на каждый позитивный триггер; agentregress помечает регрессии	Триггер, исполнение	Перед допуском к действиям
Canary / shadow-режим	Деплой на контролируемый трафик до полного раската	Shadow: параллельное офлайн-сравнение. Canary: 1% живого трафика под наблюдением selftune 24 часа	Регрессия	Перед каждым релизом с действиями

Таблица 1. Набор инструментов оценки: пять взаимодополняющих паттернов тестирования, покрывающих всю поверхность отказов навыков агентов.

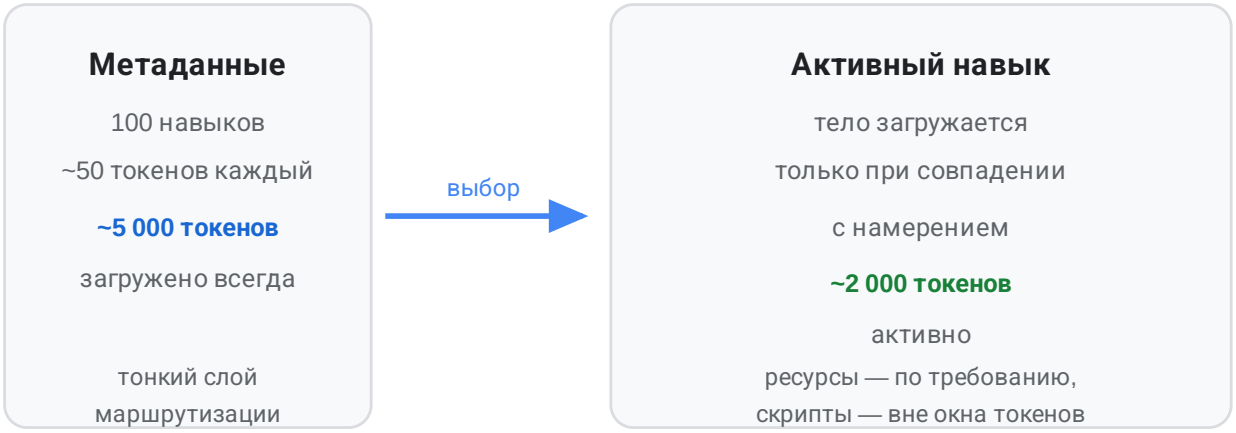


Рис. 2. Механизм гейткипинга: метаданные действуют как тонкий слой маршрутизации, удерживая число активных токенов низким, пока конкретные сигналы активации не совпадут с намерением пользователя.

Триггер — первый шлюз

Навык, который никогда не срабатывает, не может помочь. Навык, срабатывающий слишком широко, подмешивает нерелевантный контекст. Прод-анализ Vercel выявил 56% невызовов у навыков, ожидаемых к стабильной активации. Хуже того: навык, лишённый инструкций, набрал 58%, тогда как агент без навыка — 63%. Дефицит в пять пунктов показывает: плохо спроектированный навык способен активно *вычитать* возможности.

В том же исследовании Vercel отметил: пассивный индекс `AGENTS.md` с конвенциями проекта достиг 100% прохождения против базовых 53%. Это подтверждает: навыки стоит приберегать для узких, ориентированных на действие процессов, тогда как глобальный контекст должен оставаться в пассивной, всегда доступной документации.

Чтобы выйти на индустриальный стандарт точности триггера в 90%, описание в `SKILL.md` — единственное, что модель видит при маршрутизации, — должно пройти четыре проверки:

- **Тестируемая конкретность:** напишите 3 позитивных и 3 негативных триггера.
- **Однозначность:** неоднозначные запросы не пересекаются с соседними навыками.
- **Точность исполнения:** описание отражает реальное поведение, а не желаемое.
- **Устойчивость к перефразированию:** маршрутизация стабильна при любой формулировке намерения.

Качество вывода и траектория инструментов

Когда навык сработал, тестируйте отдельно итоговый вывод (что агент говорит) и траекторию инструментов (что агент делает).

Умный способ — разработка через оценку (Evaluation-Driven Development, EDD): инвертируйте процесс и напишите три JSON-кейса (вход, ожидаемые инструменты, ожидаемый вывод) ещё до черновика `SKILL.md`. Это заставляет зафиксировать чёткую функциональную спецификацию заранее. При скоринге через LLM-судью в масштабе помните две незыблемые вещи: меняйте местами эталонный и фактический выводы, чтобы устранить предвзятость порядка, и калибруйте по человеческим оценкам до согласия в 90%.

Анализ Latitude (март 2026) показал: скоринг только финального вывода пропускает на 20–40% больше кейсов, чем скоринг с учётом траектории. Разрыв образуют случаи, когда агент пришёл к правильному ответу через неверную последовательность вызовов инструментов. В read-only сценариях это допустимо. В навыках с действиями — критично: неверная траектория способна вызвать необратимые побочные эффекты.

Фреймворк оценки Google ADK предлагает три режима скоринга траектории: EXACT (точный порядок), IN_ORDER (упорядоченное подмножество) и ANY_ORDER (неупорядоченное подмножество). Валидация траектории должна соответствовать уровню навыка: для read-only навыков допустим ANY_ORDER, навыки с действиями требуют IN_ORDER или EXACT.



Три JSON-кейса (вход, ожидаемые инструменты, ожидаемый вывод) пишутся до тела навыка — это вскрывает неоднозначности описания и ошибки траекторий до их накопления.

Рис. 3. Путь инверсии: вместо «сначала код» процесс заставляет определить ожидаемые траектории инструментов и рубрики оценки самым первым шагом.

Система против навыка: иллюзия оценки

Тестирование траектории оценивает составную систему — хост-агента, взаимодействующего с навыком, — а не навык в изоляции. Когда траектория с несколькими навыками падает, часто невозможно отделить маршрутизацию агента, качество инструкций и точность исполнения.

Чтобы упростить калибровку, оценивайте навыки по паттерну «субагент с одним навыком» (агент + 1 навык против базового агента); сложную совместную загрузку нескольких навыков приберегите для продвинутого прод-стейджинга.

EDD инвертирует процесс: три JSON-кейса (вход, ожидаемые инструменты, ожидаемый вывод) пишутся до черновика `SKILL.md`, что заставляет зафиксировать чёткую функциональную спецификацию заранее. Минимальный кейс выглядит так:

```
{
  "case_id": "refund_dup_charge_001",
  "input": "Меня дважды списали за заказ #4521 во вторник",
  "expected_skill": "refund_processor",
  "expected_tool_calls": [
    {"tool": "lookup_order", "args": {"order_id": "4521"}},
    {"tool": "check_duplicate_charge", "args": {"order_id": "4521"}}
  ],
  "expected_output_format": "confirmation_with_refund_id",
  "rubric": ["подтверждает дубль", "указывает id заказа", "даёт следующий шаг"]
}
```

Фрагмент 3. Минимальный JSON-кейс оценки в EDD: входные параметры, ожидаемые траектории инструментов и рубрики заданы заранее, явно.

Три таких кейса, написанных заранее, вскрывают неоднозначности описания и ошибки траекторий до того, как они накопятся в теле навыка.

При скоринге через LLM-судью в масштабе помните две незыблемые вещи: переставляйте эталонный и фактический выводы местами для устранения предвзятости порядка и калибруйтесь по человеческим оценкам до согласия в 90%.

Бюджет токенов: изоляция — ловушка

Никогда не оценивайте навык в чистой изоляции. Агенты в проде одновременно загружают от 5 до 15 навыков. Тело навыка свыше 5 000 токенов может отлично работать в одиночку, но вызовет гниение контекста при совместной загрузке.

Ловушка составной оценки: навык против агента. Тестирование траектории оценивает составную систему — навык и хост-агента вместе. Если тест падает, не переусложняйте `SKILL.md` под конкретную модель: это разрушает переносимость. Вместо этого отделите логику исполнения от маршрутизации через «двухуровневый каркас проверок»: валидируйте код инструментов независимо, а триггеры `SKILL.md` аудируйте на разных семействах моделей, чтобы поймать хрупкие описания, привязанные к конкретной архитектуре.

MCPVerse отметила падение точности на 18,2% у Claude-4-Sonnet из-за разрастания числа инструментов и конкуренции внимания внутри контекста. Схожим образом Chroma Research

(2025) обнаружила: все передовые модели деградируют по мере роста входа — особенно когда им мешает шум совместно загруженного контента.

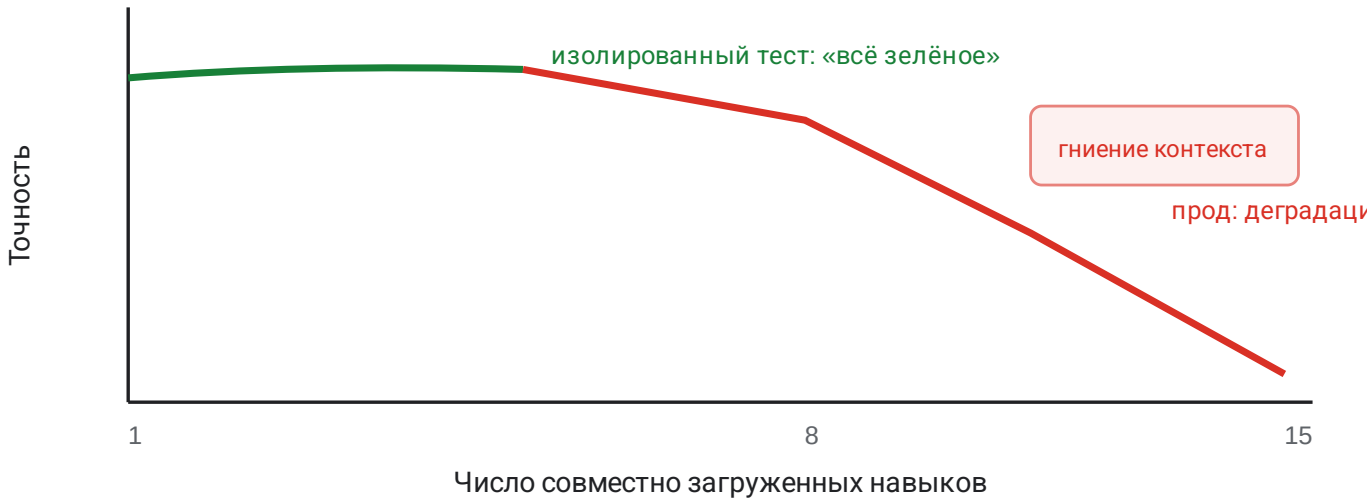


Рис. 4. Разрыв между одиночным навыком и 15 совместно загруженными. Кривая объясняет, почему прохождение изолированного теста — ложноположительный сигнал готовности к проду.

Поэтому навыки должны проходить строгие уровни полномочий:

- **Read-Only:** оценка LLM-судьёй; 90% точности триггера.
- **Draft-Only (с рецензией человека):** golden-датасет из 20+ кейсов; одобрение человека.
- **Action-Allowed:** полный состязательный red-teaming; устойчивый успех на многих прогонах (не единственная удача); ни одного отката; устойчивое pass^k.

pass^k измеряет последовательный, а не случайный успех: оценка прогоняется k раз, и успех требуется на каждом прогоне. На tau-bench (Yao et al., 2024) GPT-4o набрал 61% на pass^1, но упал ниже 25% на pass^8 — единичный успех плохо предсказывает прод-надёжность.

При калибровке этих порогов критичны два фактора:

1. **Прод-деградация:** ReliabilityBench показывает, что в проде производительность обычно падает на 20–30% относительно офлайн-бенчмарка pass@1.
2. **Предвзятость симуляции:** оценки на симуляциях могут страдать оптимистичной предвзятостью до 9% (эффект «Lost in Simulation»).

Следовательно, рецензия человеком репрезентативных выводов остаётся финальным сигналом валидации для допуска к действиям.

Что означает «полнота оценки»

Навык достигает полной оценки, удовлетворяя четырём условиям, напрямую сопоставленным с основными режимами отказов:

- **Отказ триггера:** проверка поведения триггера как позитивными (должен сработать), так и негативными (не должен) кейсами.
- **Отказ исполнения:** корректные выводы на репрезентативном диапазоне ожидаемых входов.
- **Регрессия:** добавление навыка даёт ноль падений производительности во всей существующей библиотеке.
- **Отказ бюджета токенов:** размер токенного следа навыка ограничен так, чтобы он не ухудшал производительность на несвязанных ходах.

Этот чеклист управляет повышением уровня: провал любого условия удерживает навык на уровне draft, каким бы блестящим ни был результат на счастливом пути. После верификации навык и его набор оценок готовы к прод-деплою (подробнее — в разделе 5).

5. От прототипа к продакшену

Разделы 1–4 охватили, что такое навыки, как их писать и как оценивать. Этот раздел — о том, что меняется, когда вы ставите рабочий прототип перед реальным заказчиком. Коротко: модель перестаёт быть самой интересной частью, а навыки становятся инженерным примитивом, который позволяет надёжно выпускать продукт.

Agents CLI от Google в составе Agent Platform — это командная строка и пакет навыков для сборки, оценки и развёртывания ИИ-агентов на Google Cloud. Агенты строятся на Google Agent Development Kit (ADK), а Agents CLI берёт на себя всё вокруг: скаффолдинг, оценку, деплой и наблюдаемость.

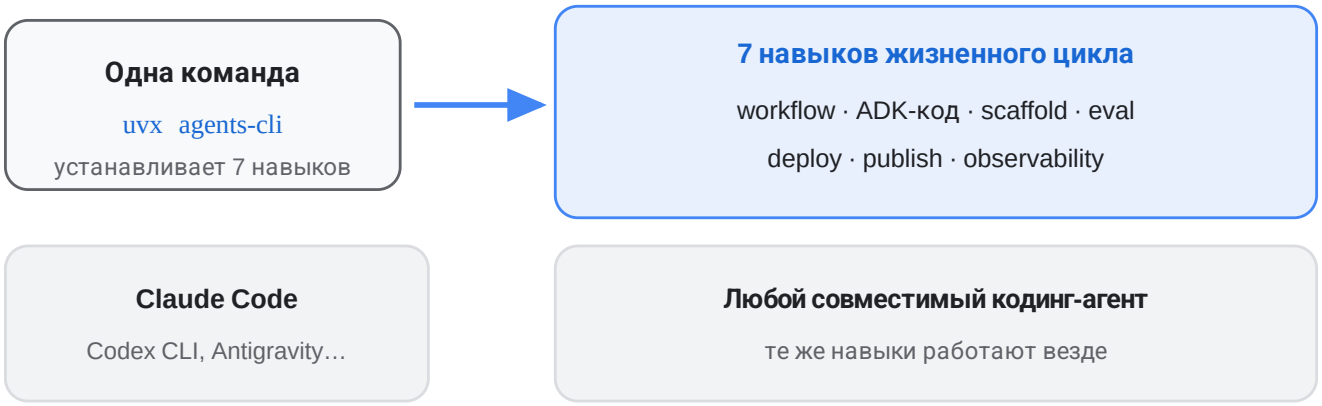


Рис. 5. Поток установки Agents CLI. Одна команда uvx ставит семь навыков в существующий кодинг-агент разработчика, покрывая весь жизненный цикл агента. Те же навыки работают в Claude Code, Codex CLI, Antigravity и любых других совместимых агентах.

Рабочий пример указывает на три свойства, которые обобщаются за пределы настройки Google:

- **Экспертиза живёт в навыках, а не в рантайме.** Рантайм коммодитизирован; семь навыков — долговечный актив.
- **Пакет навыков сочетается с тем, что вы уже используете.** Установите навыки — и ваш существующий инструмент для кодинга получит новые возможности. Того же паттерна стоит придерживаться и внутри: возможности, которые встраиваются в существующий инструментарий, а не очередной портал.

- **Весь жизненный цикл поставляется как навыки.** Scaffold, build, evaluate, deploy, publish, observe. Каждый этап, для которого раньше требовался отдельный инструмент, теперь укладывается в формат навыков.

Что на самом деле внутри агентного рантайма

Под капотом фреймворков агентный цикл сошёлся у всех вендоров: рантайм поддерживает диалог, вызывает модель, исполняет инструменты, читает файлы, возвращает ответ. Поразительно, как мало кода внутри таких рантаймов относится к рассуждению.

Недавний reverse-engineering Claude Code v2.1.88 (Liu, Zhao, Shang и Shen, 2026) показал: 98,4% кодовой базы — это операционная инфраструктура: классификаторы разрешений, конвейеры уплотнения контекста, делегирование субагентам, хранение сессий — и лишь 1,6% приходится на сам агентный цикл. Модель живёт за удалённым API; инженерия вокруг неё и делает систему прод-класса. Сайт-компаньон ccunpacked.dev визуально картографирует ту же архитектуру.

Это архитектурное прозрение стоит за всем дальнейшим: по мере того как базовые модели сходятся по уровню рассуждений, дифференциатором автономной надёжности становится детерминированная инженерия вокруг модели, а внутри неё — навык как единица, которую компонуют и переиспользуют.

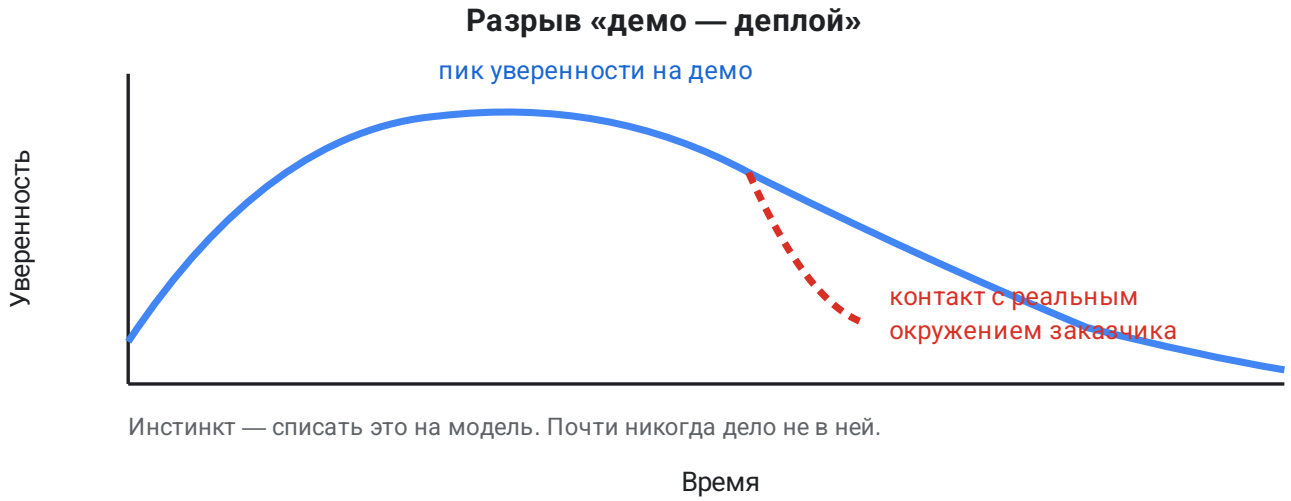


Рис. 6. Разрыв «демо — деплой». Уверенность команды рано достигает пика, затем рушится при контакте с реальным окружением заказчика. Инстинкт — назвать это проблемой модели; почти всегда это не так.

Почему навыки — единица улучшения

Наивная теория улучшения агентов: лучшие модели дают лучших агентов. В проде модель — это инфраструктура, а навыки — примитив, через который улучшения выходят в свет. Каждый

новый навык — небольшая, находящаяся в чьей-то собственности, тестируемая единица способности (как задано в разделе 1). Когда появляется новый крайний случай, достаточно отредактировать один `SKILL.md`; эффективная способность агента растёт без трудностей монолитного промпт-инжиниринга.

Три свойства навыков обеспечивают это:

- **Они условны.** Загружаются, только когда их описание совпадает с задачей.
- **Они компонуемы.** Один навык может вызывать инструменты другого или цепляться дальше по потоку, при этом ни один из них не знает о другом (раздел 7 разрабатывает историю композиции вглубь).
- **Они имеют владельца.** Каждый навык живёт в версионируемой папке с явным автором, поэтому улучшение распределено, а не упирается в центральную платформенную команду.

Сравните с альтернативами:

Стиль улучшения	Длительность цикла	Режим отказа	Кто может делать	Контекстный налог
Смена модели	Дни — недели	Регрессия в несвязанных задачах	ML/platform-команда	Нет (на весах)
Правка системного промпта	Минуты — часы	Гниение контекста, конфликт инструкций	Владелец файла промпта	Статичный (каждый ход платит)
Файнтюн	Недели — месяцы	Катастрофическое забывание, переобучение	Только ML-команда	Нет (на весах)
Новый навык	Часы — дни	Ограничен только совпадающими ходами	Любая доменная команда	Динамичный (грузится по требованию)

Таблица 2. Сравнение подходов к улучшению агентов: длительность цикла, режимы отказа, организационное владение и контекстные затраты.

Сбой, ломающий демо: переполнение контекста

Самый частый режим отказа агентов в проде — не галлюцинация. Это **переполнение контекста**: модель получает больше контекста, чем способна эффективно использовать, и тихо деградирует, прежде чем оператор заметит. Две линии исследований это обосновывают:

Lost in the Middle (Liu et al., TACL 2024). В мультидокументном QA и поиске производительность максимальна, когда релевантная информация стоит в начале или конце входа, и падает в

середине; это U-образная кривая, верная даже для моделей, обученных на длинных контекстах.

Гниение контекста (Chroma Research, 2025). На 18 передовых моделях — Claude 4 Opus и Sonnet, Gemini 2.5, Qwen3 — производительность падает по мере роста входа, даже при неизменной сложности задачи. Каждая модель становится хуже, и тем быстрее, чем труднее отличить релевантный контент от дистракторов. Шум типичных агентских контекстов — выводы инструментов, полурелевантные результаты поиска, промежуточные рассуждения — среди худших.

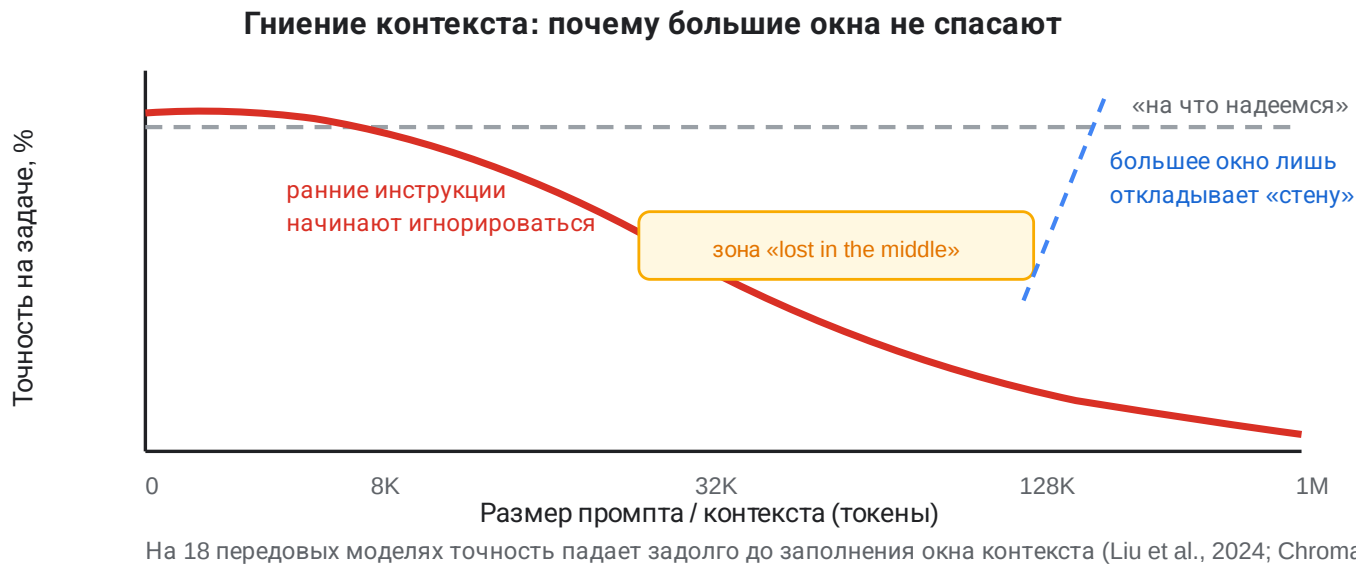


Рис. 7. Гниение контекста на практике: с ростом промпта точность на фиксированной задаче падает задолго до заполнения окна. Пунктир — наивное ожидание; кривая — реальное поведение 18 передовых моделей.

Что это значит для бюджета токенов

Прогрессивное раскрытие из раздела 2 — архитектурный ответ: метаданные каждого навыка грузятся на старте, тело навыка — только при совпадении описания, вспомогательные файлы — только когда тело на них ссылается.

Математику стоит показать. Возьмём агента с пятьюдесятью различными процессами. Как единый системный промпт он загружает 15 000 токенов на каждом ходу. Как библиотека навыков — около 4 000 токенов описаний плюс тело одного активного навыка примерно на 2 000 токенов, то есть ~6 000 токенов суммарно; остальные сорок девять тел лежат на диске. Anthropic публиковала примеры, где перевод процесса в навыки сокращал активный контекст с примерно 150 000 токенов до 2 000 — более чем на 98 процентов.



Рис. 8. Токенная экономика: один большой промпт против библиотеки из пятидесяти навыков. Библиотека держит пятьдесят единиц способности доступными, но в контексте в любой момент лишь активное тело.

Отсюда три практических следствия:

- **Вместимость — неверная метрика.** Окно в 1M токенов может заметно деградировать уже на 50K.
- **Активный контекст — это бюджет, а не сосуд.** Каждый токен перед моделью отнимает внимание у остальных. Относитесь к системному промпту так, как инфраструктурные команды относятся к памяти: это конечный ресурс, распределяемый осознанно.
- **Навыки снимают ограничение.** Они держат активный контекст малым, оставляя доступную способность практически безграничной.

Как только у команды есть рабочая библиотека, вопросы смещаются с поддержки одного навыка на эволюцию, композицию и более широкую экосистему.

6. О мета-навыках и самоулучшающихся навыках

До сих пор каждый навык в этом документе написан человеком. Эксперт садится, набрасывает `SKILL.md`, тестирует, выпускает. Это правильная точка старта. Но когда у вас есть рабочая библиотека, напрашивается следующий вопрос: может ли агент помогать писать, оценивать и улучшать навыки?

Это территория мета-навыков: навыков, чья задача — создавать, оценивать или улучшать другие навыки. На практике они делятся на четыре корзины:

1. **Авторство.** Навыки, которые берут описание процесса и производят черновик `SKILL.md`. В ADK от Google есть паттерн «skill factory», делающий это через SkillToolset. Anthropic поставляет навык skill-creator, который проводит через создание, оценку и настройку.
2. **Авторство по трассировкам.** Вместо описания процесса человеком — наблюдайте, как агент успешно делает это несколько раз, а затем превратите трассировку в навык. Процесс skill-creator напрямую поддерживает это через сбор по трассировкам. Задача человека смещается с написания навыка на подтверждение того, что собранная версия зафиксировала верные шаги.

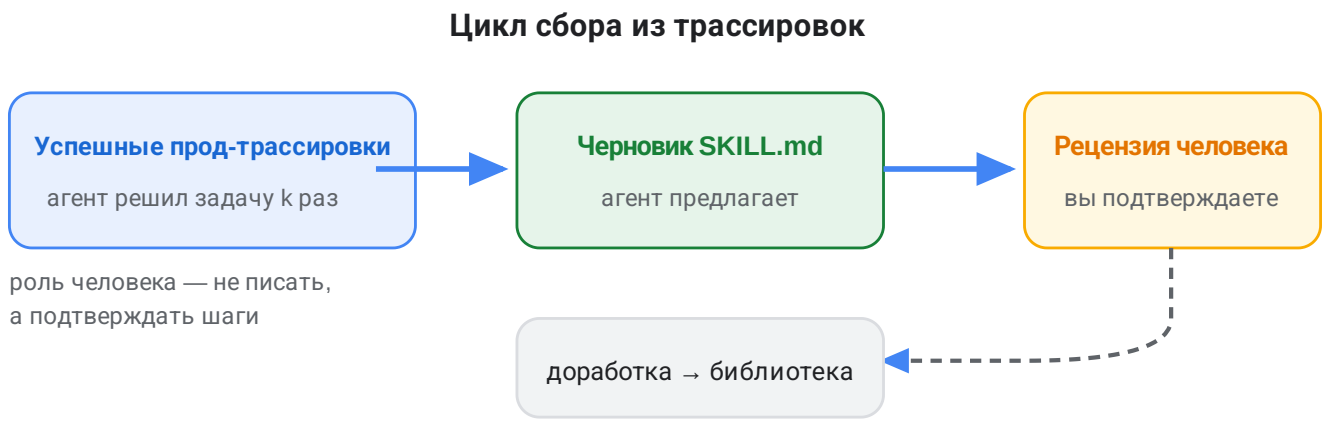


Рис. 9. Цикл по шагам: реальные успешные прод-истории превращаются в надёжные процедурные памяти без ручного написания.

3. **Улучшение.** Навыки, которые берут существующий навык плюс набор падающих кейсов оценки и предлагают правки. SkillOptimizer от Сабу и цикл оптимизации описаний от Anthropic — два примера. Ещё один — паттерн autoresearch Карпати: агент предлагает изменение целевого файла, запускает ограниченный эксперимент и сохраняет изменение только при улучшении метрики.

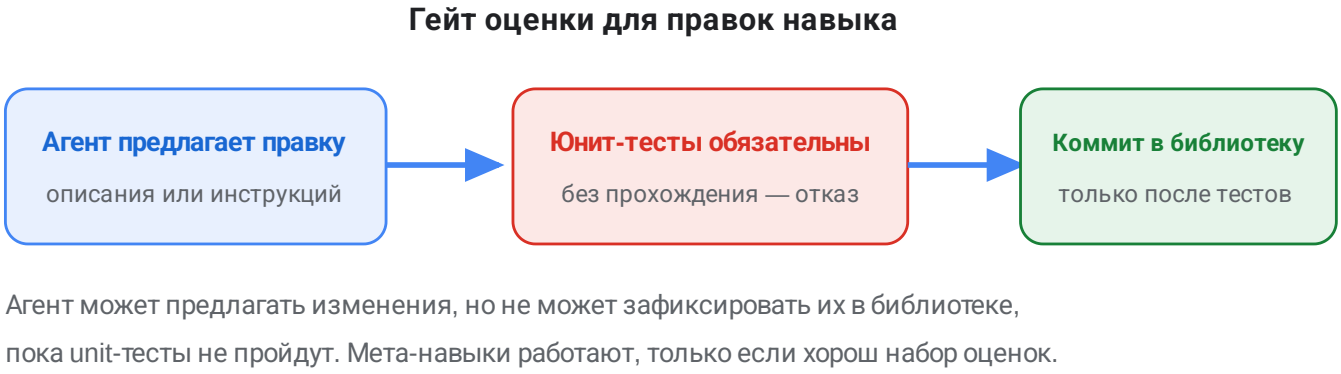


Рис. 10. Гейт оценки: агент вправе предлагать изменения описаний и инструкций, но не может зафиксировать их в библиотеке, пока не пройдут юнит-тесты.

4. **Эволюция библиотеки.** Навыки, которые со временем расширяют библиотеку — так Voyager выращивал свою библиотеку навыков Minecraft. Агент заканчивает задачу, для которой у него не было навыка, замечает, что только что решил повторяющуюся проблему, и предлагает добавить новый навык. self-learning-skill Шмида — сообщественная эталонная реализация этого паттерна.

Где это ломается

Мета-навыки работают, только если хорош ваш набор оценок. Агент, которому позволено редактировать собственные навыки, с радостью оптимизирует любую метрику, на которую вы укажете, — включая метрики, которые легко нагеймить. Оценочная работа из раздела 4 — то, что держит этот процесс честным. Без солидных тестов точности триггеров, регрессионных тестов и человеческих выборочных проверок автономный цикл улучшения будет тихо ухудшать библиотеку, отчитываясь, что она становится лучше.

Несколько привычек, которые выдержали проверку временем:

- **Всё, что пишет агент, попадает в библиотеку на уровень draft**, независимо от того, насколько уверен мета-навык. Дальше — та же лестница Read / Draft / Act из раздела 4, что и для человеком написанных навыков.

- **Первые несколько правок — с человеком в цикле.** Даже когда метрика явно улучшилась, просмотрите diff. Ошибка, которую совершает агент (переподгонка описания под несколько тест-кейсов, поломка нижестоящего навыка, о котором он не знал), — ровно та, которую человек ловит за тридцать секунд.
- **Не начинайте с мета-навыков.** Сначала запустите ручной цикл авторства. Самый быстрый путь к плохой библиотеке — навести агента на пустую папку и попросить сгенерировать пятьдесят навыков.

Куда это движется

Паттерн кристаллизуется примерно так: люди пишут первую версию каждого навыка, мета-навыки берут на себя повторяющуюся часть сопровождения (настройка описаний, добавление тест-кейсов, выявление регрессий), а небольшая подгруппа команд экспериментирует с полным саморасширением. Наиболее интересный фронтир — третья категория, где агент предлагает новые навыки на основе того, что видит в прод-трафике. Это многообещающе. И это же место, где всё ломается быстрее всего, если оценочные гейты не затянуты.

Практическая версия этого — в Приложении A: шпаргалка покрывает, что делать (и чего не делать), когда вас тянет позволить агенту писать собственные навыки.

7. Композиция и упаковка навыков

Реальные рабочие процессы не помещаются в один навык. Проблема композиции — как навыки ссылаются друг на друга, передают состояние и избегают циклических зависимостей.

Передача сырых выводов LLM между изолированными навыками в монолитной системе неэффективна: состояние запутывается, исполнение становится недетерминированным, отладка усложняется. Архитектура агентов эволюционировала от наивной склейки промптов к предсказуемой оркестрации.

Маршрутизация исполнения: DAG-оркестрация

Ранние архитектуры оказались хрупкими и подверженными накоплению ошибок, когда первые стадии галлюцинировали. Индустриальное решение — оркестрация с помощью направленного ациклического графа (DAG).

- **Разделение состояния:** маршрутизация состояния в DAG-архитектуре не опирается на накопление истории исполнения внутри промпта LLM.
- **Файловая шина сообщений:** DAG-контроллер оркеструет передачи, пропуская между узлами-субагентами структурированные ссылки на схемы.
- **Защита внимания:** абстрагирование полезной нагрузки от текстового входа модели предотвращает раздувание окна контекста и сохраняет ёмкость модели.

Упаковка окружения: профили возможностей

Активация каждого навыка подряд ухудшает естественно-языковую маршрутизацию и перегружает окно контекста. Архитекторам стоит использовать средства управления «профилями возможностей» (Capability Profiles) — специализированными персонами под конкретные состояния исполнения. Профиль — модульный набор инструментов, определяющий:

- активные навыки и доступ к инструментам;
- системные инструкции и операционные предохранители;
- автоматизированные процессы и топологии субагентов;
- параметры LLM: выбор модели и температуру.

Во время исполнения оркестрационный слой выгружает прежние системные инструкции и сбрасывает устаревшие переменные, прежде чем загрузить новый профиль в память. Этот

строгий демонтаж и пересборка предотвращают потерю контекста.

Наполнение графа: каноническая таксономия навыков

Чтобы построить DAG, дискретные инженерные способности отображаются на конкретные функции узлов в графе исполнения:

- **Генератор:** превращает намерение пользователя в структурированные артефакты.
- **Ревьюер и шлюз:** детерминированные ворота, блокирующие исполнение при провале валидации.
- **Пайплайн:** оркестрирует линейные пути внутри более широкого DAG-окружения.
- **Инверсия и восстановление:** заставляет агента уточнить предположения до исполнения.
- **Обёртки доменного контекста:** узлы-справочники, обучающие конвенциям предметной области.

Контекстный долг и смещение интеллекта влево

Навыки сжигают внимание модели — дефицитный ресурс. Когда авторы пытаются добиться детерминированного поведения в рантайме, раздувая описания навыков (например, «ВСЕГДА ДЕЛАЙ X»), они накапливают **контекстный долг**. Модели учатся игнорировать такие капс-императивы — ровно как разработчик игнорирует стену нечитаемых предупреждений.

Инженерный бест-практис — **смещать интеллект влево**. Вместо надежды, что LLM верно истолкует сложные правила в рантайме, дистиллируйте субъективные суждения в навыки. Выталкивая логику из промпта LLM в стандартные тестируемые скрипты, вы сокращаете хаотичную поверхность приложения.

Архитектурные компромиссы

Архитектура	Механизм	Главное преимущество	Лучше всего для
Линейные пайплайны	Последовательная передача текста между фиксированными узлами	Низкие инженерные накладные, быстрый прототип	Однодоменные генеративные задачи низкой сложности
DAG-оркестрация	Графовое параллельное исполнение с файловой шиной состояния через ссылки на схемы	Защита от циклов, строгая изоляция контекста	Мультиагентные процессы с высокой надёжностью
Профили возможностей	Сменные версионируемые наборы параметров и инструментов	Быстрая смена персон с очисткой памяти жизненного цикла	Ролевой деплой и доменные агенты

Таблица 3. Архитектурные компромиссы: линейные пайплайны, DAG-оркестрация и профили возможностей в мультиагентных системах навыков.

Действенные бест-практисы

- **Пишите программное обеспечение, а не правила:** заменяйте негативные LLM-инструкции детерминированными программными ограничениями, делающими невалидные действия невозможными.
- **Внедряйте прогрессивное раскрытие:** загружайте сложные инструкции динамически, только когда навык явно вызван.
- **Разделяйте состояние:** никогда не используйте окно контекста LLM как базу данных. Передавайте субагентам только URI или указатели через файловую систему или шину сообщений.

8. Как выбрать среди сотен существующих навыков

К началу 2026 года публичные маркетплейсы навыков пересекли отметку в 40 000 объявлений, а ведущая платформа сообщала о десятках тысяч новых навыков только за первые недели января. На Google Cloud Next 2026 Google запустил официальный репозиторий навыков на github.com/google/skills — установка через `npx skills install github.com/google/skills` для Antigravity CLI и любого другого совместимого кодинг-агента. Репозиторий навыков Anthropic, библиотека навыков Google ADK, официальный репозиторий Google и сообщественные маркетплейсы вроде awesome-llm-apps теперь содержат больше навыков, чем любой практик в силах просмотреть лично. Проблема выбора реальна и растёт.

Помогают три эвристики.

1. **Для вендор-специфичных инструментов предпочитайте first-party навыки.** Навык BigQuery от Google, официальный навык Stripe — то, что написано людьми, построившими базовую систему. Такие навыки точнее и лучше поддерживаются, чем сообщественные альтернативы.
2. **Пините всё, от чего зависите.** Сообщественные навыки развиваются, и незакреплённая зависимость, работавшая вчера, может отказать завтра.
3. **Аудируйте до принятия.** Навык — это код, работающий в вашем контексте. Относитесь к нему как к любой другой зависимости, с той же гигиеной цепочки поставок.

Не все источники равны. В начале 2026 существуют три категории источников навыков, и правильная операционная позиция для каждой своя:

Источник	Дефолт доверия	Примеры	Кто поддерживает
First-party вендорские навыки	Доверие по умолчанию; закрепить версию	google/agents-cli, google/skills, google-gemini/gemini-skills, anthropics/skills, stripe/ai, microsoft/skills	Команда, построившая базовый продукт
Организационно курируемые навыки	Доверие внутри организации; ревью при принятии	your-org/retail-skills, your-org/finance-skills (приватные, внутренние)	Ваши доменные команды, с ревью через PR
Сообщественные навыки	Аудит до принятия; агрессивно пинить	VoltAgent/awesome-agent-skills, SkillsMP, addyosmani/agent-skills, отдельные GitHub-репозитории	Авторы-волонтёры, разный уровень поддержки

Таблица 4. Источники навыков агентов: дефолты доверия, официальные примеры и владение сопровождением.

9. Заключение

Мы начали этот отчёт с удивительно простой концепции: папка с markdown-файлом и парой необязательных скриптов. И всё же эта лёгкая структура — навык агента — фундаментально меняет то, как мы строим ИИ-агентов. Она наконец даёт фундаментальным моделям настоящую, тестируемую процедурную память, позволяя им помнить, как выполнять задачи пошагово. Опираясь на магию прогрессивного раскрытия, навыки решают проблему гниения контекста. Один универсальный агент теперь может беспрепятственно получать доступ к множеству специализированных процессов, не захлёбываясь в своём бюджете токенов.

Паттерн, к которому мы всё время возвращаемся: формат намеренно мал, чтобы интересная работа происходила *вокруг* него, а не внутри. Оценка навыков в реалистичных условиях совместной загрузки — интересна. Композиция навыков в процессы без использования окна контекста как шины сообщений — интересна. То, что агенты чертят навыки из успешных трассировок, а люди рецензируют вместо авторства, — интересно. Кодирование двадцати лет институциональных знаний в версионируемую, тестируемую, управляемую библиотеку — интересно. Всё это теперь достижимо так, как не было ещё двенадцать месяцев назад, — потому что примитив существует.

На протяжении документа мы старались быть конкретными: что устоялось, что ещё формируется, что, вероятно, изменится. **Формат устоялся:** agentskills.io — теперь открытый стандарт, принятый всеми значимыми кодинг-агентами, ИИ-чат-ботами и агентными фреймворками. **Архитектура вокруг ещё формируется:** оценка при совместной загрузке, оптимизация на уровне библиотеки, создание навыков агентом и паттерны управления, делающие всё это безопасным в масштабе.

Если вы начинаете сегодня, наша рекомендация — та же, что звучала на протяжении всего текста: начинайте с малого, начинайте со знаний, которые у вас уже есть, относитесь к навыкам как к коду, измеряйте то, что выпускаете, и не тянитесь к мультиагентной архитектуре, когда достаточно навыков. Команды, разобравшиеся сейчас, построят более чистые системы, чем те, что дождутся индустриального консенсуса.

> Формат устоялся. Работа только начинается.

Приложение A — Практическая шпаргалка

Этот раздел задуман как печатный и автономный. Он сжимает остальной отчёт в решения, с которыми столкнётся команда.

Минимальный SKILL.md

Копируйте, замените плейсхолдеры. Готово.

```
---
name: skill-name
description: |
  [Что делает, одним предложением с глаголом.] Используй этот навык,
  когда пользователь [триггер 1], [триггер 2] или [триггер 3].
  НЕ используй для [анти-триггер 1] или [анти-триггер 2].
version: 1.0.0
license: MIT
allowed-tools: [Необязательно] Read Bash Write
metadata:
  author: [Необязательно] ваш-хэндл
---
# Имя навыка
## Когда использовать
- [Конкретный сценарий]
- [Конкретный сценарий]
## Когда НЕ использовать
- [Сценарий вне границ]
## Порядок действий
1. [Шаг]
2. [Шаг]
3. См. `references/advanced.md` для [крайнего случая].
## Примеры
- Вход: «...» → Выход: «...»
## Формат вывода
- Используйте `assets/template.md` и т.п.
## Анти-паттерны
- Не делайте [...]
```

Структура папки

```
skill_name/
├── SKILL.md          # Обязательно: YAML-frontmatter + markdown-инструкции
├── scripts/          # Необязательно: исполняемые скрипты-помощники (Python, Bash)
├── references/       # Необязательно: дополнительный контекст, грузится по требованию
├── assets/           # Необязательно: файлы для вывода (шаблоны, ресурсы)
├── ...               # Любые дополнительные файлы и директории
```

Именование

- Имя директории: snake_case
- Имя навыка: kebab-case
- Предпочитайте герундий: processing-pdfs, а не pdf-processor
- Избегайте общих имён: helper, utils, tools, data
- Избегайте вендорских префиксов: claude-*, gemini-*, anthropic-*
- Избегайте внутреннего жаргона, незнакомого посторонним

Поле description

Описание — это алгоритм маршрутизации. Потратьте на него больше времени, чем на что-либо ещё.

- Укажите, что навык делает И когда его использовать.
- Выносите триггерные ключевые слова в начало («Сгенерируй сообщение коммита…», а не «Этот навык помогает с…»).
- Включайте «когда НЕ использовать», чтобы предотвратить избыточное срабатывание.
- Будьте настойчивы, если модель срабатывает неохотно.
- ≤200 символов для API; ≤1024 в YAML. Большинство авторов целятся примерно в 50 слов.

Пять правил

1. **Один навык — одна работа.** Если не можете описать навык одним предложением — это два навыка. Декомпозируйте до написания.
2. **Описания — это интерфейс.** Агент выбирает навыки, читая описания. Расплывчатое описание значит неиспользуемый навык.

3. **Навыки — это зависимости.** Относитесь к ним как к библиотекам: версионируйте, пините, ревьюйте в PR. Навык без теста — надежда, а не способность.
4. **Правильная команда владеет правильным навыком.** Доменные эксперты владеют доменными навыками. Не позволяйте ИИ-команде стать бутылочным горлышком для доменных знаний организации.
5. **Агентный рантайм взаимозаменяем.** Не привязывайте навыки к одному рантайму. Переносимость — часть ценности.

Принципы качества

1. **Сначала выполните задачу сами.** Реальный провал даёт сигнал. Спекуляция даёт шум.
2. **Давайте причину, а не только правило.** Модели прекрасно обобщают на крайние случаи, когда понимают, почему инструкция существует. Заметили у себя «ВСЕГДА» или «НИКОГДА» капсом — остановитесь и объясните рациональность вместо этого.
3. **Каждая строка должна заслужить своё место.** Держите подводные камни, точные команды, бизнес-логику, анти-паттерны. Убирайте шаблонное вроде «всегда валидируй вывод», которое модель и так знает.
4. **Один навык — одна работа.** Если для описания нужен союз «и» между несвязанными возможностями — разделите.
5. **Делайте инструкции проверяемыми.** Если агент не может определить, следовал ли он правилу, правило слишком расплывчато.
6. **Бандлите повторяющееся.** Хелпер-код, который агент постоянно пере-выводит, — место в `scripts/`.

Что делать и чего не делать

Делать:

- начинать с малого и конкретного;
- тратить непропорционально много времени на описание — это алгоритм маршрутизации; всегда включайте: что делает, когда использовать, когда не использовать;
- доверять прогрессивному раскрытию;
- бандлить детерминированную работу в `scripts/`, а не в инструкции;
- относиться к навыкам как к коду;
- помнить: навыки + MCP, а не навыки против MCP;
- иметь тест-кейсы и контролируемую программную оценку (раздел 4 — как именно).

Не делать:

- писать расплывчатые описания вроде «помогает с документами» — используйте конкретные глаголы, триггерные фразы и клаузу «когда не использовать»;
- писать тела `SKILL.md` длиннее 5 000 слов — уносите детали в `references/`;
- хардкодить пути или секреты;
- встраивать правила «всегда делай X», больше подходящие для `AGENTS.md`;
- устанавливать недоверенные сторонние библиотеки или навыки без сканирования;
- изобретать MCP заново в виде скриптов.

Запахи навыков (пересмотрите, если видите)

- Свыше 5 000 слов. Вероятно, это два навыка или справочный материал для `references/`.
- Две доменные команды могли бы им владеть. Ещё не декомпозирован. Разделите по границам команд.
- Не можете написать три тест-кейса. Описание слишком расплывчато; навык делает слишком много.
- Не ссылается ни на какой другой ресурс. Возможно, это просто длинная инструкция для системного промпта.
- Вы постоянно добавляете секции «крайние случаи». Вероятно, каждый крайний случай хочет собственный навык.
- Описание начинается с «полезный навык для…». Перепишите: описание должно называть триггер, входы и выход.

Чеклист полноты оценки

Навык «оценён», только когда выполнены все четыре:

- ☐ **Триггер.** Позитивные и негативные тест-кейсы. Цель — 90% точности триггера.
- ☐ **Исполнение.** Корректные выводы на репрезентативном диапазоне входов.
- ☐ **Регрессия.** Добавление навыка даёт ноль падений в существующем наборе библиотеки.
- ☐ **Бюджет токенов.** При совместной загрузке с 5–15 часто активными навыками не ухудшает несвязанные ходы.

Любой провал удерживает навык на уровне draft, независимо от результатов на счастливом пути.

Чеклист деплоя

- ☐ Frontmatter валиден (линт проходит).
- ☐ Описание включает что + когда + когда-не.
- ☐ У скриптов есть юнит-тесты, проходящие в CI.
- ☐ Набор оценок проходит в CI с минимальным порогом.
- ☐ Скан безопасности чист (нет секретов, нет недоверенных зависимостей).
- ☐ Описание рецензировано кем-то помимо автора.
- ☐ Если публикуете наружу — протестированы кросс-инструментальные пути установки.
- ☐ Обновлён орг-провижининг для администраторов (если применимо).

Однострочная ментальная модель

> Системный промпт = инстинкт. AGENTS.md = README проекта. Инструменты / MCP = руки. RAG = библиотека. Навыки = инструктаж, который опытный коллега выдаёт вам в первый день — и который ИИ никогда не забывает.

С чего начать завтра

1. **Возьмите самого опытного практика на час.** Попросите рассказать три рабочих процесса, которые он выполняет регулярно. Запишите.
2. **Выберите самый повторяемый.** Прогоните промпты сами, без загруженного навыка. Отметьте, где агент падает.
3. **Набросайте `SKILL.md` из транскрипта.** Напишите три кейса оценки (два позитивных, один негативный) до черновика тела.
4. **Выпустите на read-only уровень.** Тестируйте в условиях, приближенных к проду. Итерируйте описание, пока точность триггера не пробьёт 90%.

Повторяйте. Стройте библиотеку по одному процессу за раз. Сопротивляйтесь соблазну дать LLM сгенерировать пятьдесят навыков в первый день.

Приложение B — Кейс: экспертиза домена как код (вертикальные навыки в ретейле)

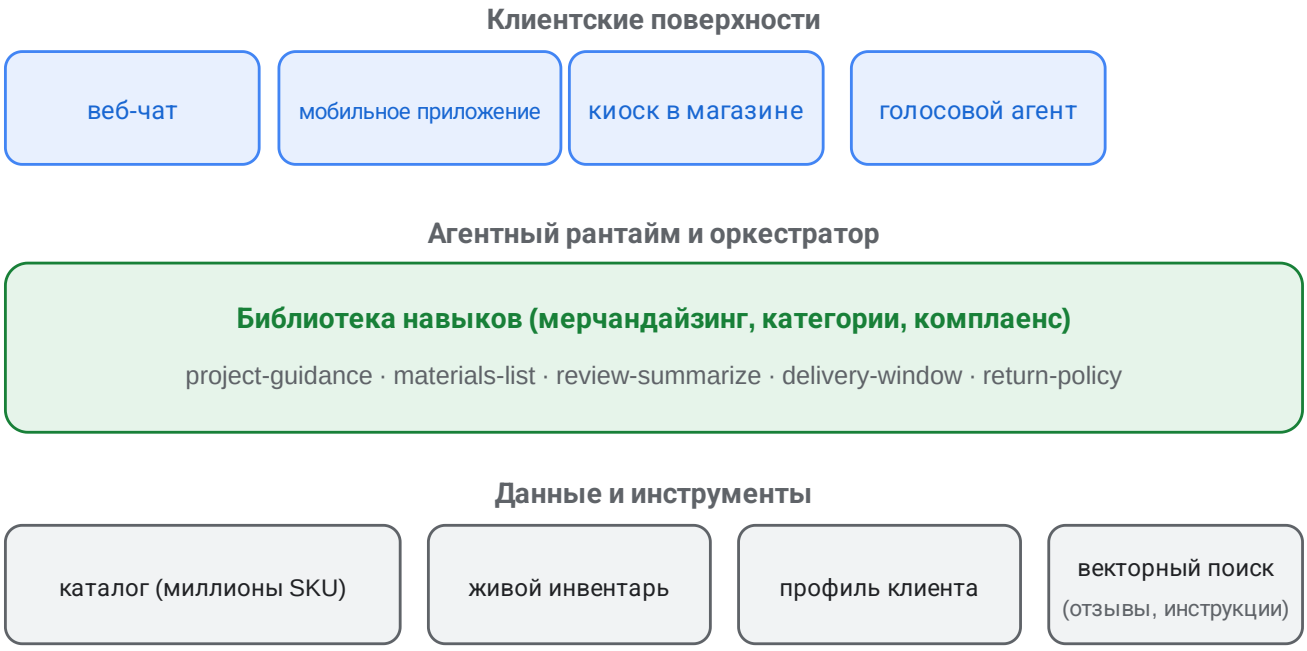
Это рабочий пример отчёта. Он показывает, как крупный ритейлер структурировал бы библиотеку навыков, чтобы захватить институциональные знания, отличающие бренд от конкурентов, и почему именно библиотека, а не какой-то кастомный агент, является долговечным стратегическим активом. Несколько крупных ритейлеров публично описывали ИИ-ассистентов с возможностями, соответствующими архитектуре ниже. Мы описываем паттерн, а не реализацию конкретного вендора.

Почему ретейл — канонический кейс для навыков

Два ритейлера на одном агентном рантайме, с похожими данными через похожие API, дадут совершенно разный покупательский опыт. Ретейл-экспертиза относится к знаниям, исторически застрявшим в трёх местах, куда ИИ-системам трудно было добраться: в головах старших байеров, мерчандайзеров и категорийных менеджеров; в тридцатистраничных операционных регламентах, которые никто не читает; и в Slack-каналах, где ответ на вопрос «стоит ли рекомендовать этот товар вместе с тем?» живёт в треде 2023 года.

Навыки захватывают это знание в форме, которую клиентские системы компании действительно используют.

Как выглядит архитектура



Инструменты снизу доступны через MCP и управляемые поисковые интеграции.

Рис. 11. Навык-первая ретейл-архитектура (иллюстративно). Клиентские поверхности лежат поверх агентного рантайма, который загружает навыки с мерчандайзинговыми, категорийными и комплаенс-знаниями. Инструменты доступны через MCP и управляемые поисковые интеграции.

В архитектуре три слоя. Верхний — клиентская поверхность: чат на сайте, мобильное приложение, киоск в магазине, голосовой агент в колл-центре. Каждая поверхность тонкая: пересылает ввод рантайму и рендерит ответ.

Средний слой — агентный рантайм и оркестратор, поддерживающий диалог, загружающий навыки, вызывающий инструменты и собирающий ответ.

Нижний слой — плоскость данных и инструментов: товарный каталог (часто миллионы SKU), живой инвентарь с данными о наличии по магазинам, профиль клиента и история заказов, база знаний проекта и векторный поиск по отзывам, инструкциям и спецификациям.

Для этого отчёта важен средний слой. Сам рантайм универсален: Google Agent Development Kit, Claude Agent SDK от Anthropic или любой другой рантайм, поддерживающий открытый стандарт навыков. Навыки, загруженные в этот рантайм, специфичны для домена ритейлера — именно они несут экспертизу бренда покупателю.

Иллюстративная библиотека навыков

Как реально выглядит такая библиотека? Ниже — репрезентативный набор навыков, которые крупный ритейлер товаров для дома, вероятно, поддерживает. Каждый навык — одна папка. У каждого один владелец. У каждого собственный набор оценок (по паттернам раздела 4). Вместе они составляют рабочую память компании о том, как она обслуживает клиентов.

- **project-guidance.** Кодирует знание мастеров-отделочников, превращающее расплывчатый запрос («как уложить плитку в душевой?») в пошаговый план. Включает структурные зависимости (основание должно быть гидроизолировано до укладки), логику порядка работ (раскрой до установки) и предупреждения о типичных ошибках. Владелец — команда знаний о работах. Уровень read-only.
- **materials-list.** Принимает описание проекта (голос, текст или частичный список) и выдаёт сгруппированную спецификацию материалов, включая позиции, которые подрядчик, скорее всего, забудет. Владелец — Pro-мерчандайзинг. Уровень draft-only: клиент проверяет перед покупкой.
- **review-summarize.** Сжимает длинные обзоры товара в плюсы, минусы и типичные сценарии использования. Срабатывает, когда клиент спрашивает о реальном опыте эксплуатации товара. Владелец — персонализация. Уровень read-only.
- **delivery-window.** Вычисляет варианты доставки «последней мили» и ориентировочное время прибытия с учётом местоположения клиента, наличия в магазине и грузовой сети компании. Владелец — fulfilment. Уровень read-only.
- **return-policy.** Кодирует правила возврата компании, включая десятки исключений для спецзаказов, опасных материалов, индивидуальных раскроев и контрактных цен. Владелец — клиентский сервис. Уровень read-only. Повышение до action-allowed (например, оформление возврата) требует второго навыка с куда более строгим ревью.

Каждый навык достаточно мал, чтобы ревьюиться, тестироваться и выпускаться независимо.

Как один и тот же запрос маршрутизируется через библиотеку

Представьте клиента: «Хочу переделать ванную в детской, что мне нужно?»

Рантайм загружает L1-описания всех навыков на старте сессии (~30–80 токенов на каждое, ~2 КБ суммарно). Он определяет, что запросу соответствует project-guidance, загружает его тело — и получает план этапов ремонта. Если клиент интересуется доставкой — загружается delivery-window. Если возвратом конкретного товара — return-policy. Прежние навыки могут оставаться в контексте или освобождаться по ходу разговора.

Заметьте, чего не происходит. Нет монолитного агента «ассистент по ремонту», обученного на всех мыслимых сценариях. Каждый кусок экспертизы попадает в контекст, только когда разговор до него доходит. Активный контекст остаётся малым. Доступная способность фактически безгранична.

Владение: кто пишет какой навык

Единственное самое важное решение управления, которое ритейлер принимает о своей библиотеке навыков, — кто владеет каждым навыком. Принцип: распределять владение командам, которые уже владеют базовой экспертизой:

Семейство навыков	Команда-владелец	Почему
project-guidance, product-fit	Знания о работах / категорийный менеджмент	Эти команды держат мерчандайзинговые и торговые правила; они и так правят их еженедельно
materials-list	Pro-мерчандайзинг	Специфичная для Pro-сегмента логика спецификаций живёт у Pro-команды
delivery-window	Операции магазина / fulfilment	Правила живого инвентаря и грузоперевозок меняются постоянно; команда, которая их ведёт, владеет и навыком
review-summarize	Персонализация / data science	Ближе всех к базовому NLP и сигналам отзывов клиентов

Таблица 5. Распределённая модель владения: семейства ретейл-навыков сопоставлены с доменными командами, отвечающими за их бизнес-логику.

Лестница read / draft / act

Второе решение управления — что каждому навыку позволено делать. Это та же уровневая модель из раздела 4, применённая с ретейл-примерами:

Уровень	Возможность	Ревью	Примеры
Read-Only	Может получать, запрашивать, описывать данные; не может менять состояние	Одобрение доменной команды	review-summarize, store-locator, project-guidance
Draft-Only	Может создавать контент для ревью человеком; не может отправлять или коммитить	Доменная команда + владелец формата	draft-customer-email, materials-list
Action-Allowed	Может выполнять необратимые операции на реальных системах	Доменная команда + security/compliance + исполнительное одобрение	issue-refund, send-customer-message, reserve-inventory

Таблица 6. Лестница управления read/draft/act: возможности навыков, требования к ревью и операционные примеры.

Это куда защитимее перед командой безопасности и перед регулятором, который рано или поздно появится, чем альтернатива: агент-чёрный ящик, делающий всё, что дают его обучение плюс системный промпт.

Почему с этим сложнее конкурировать, чем с кастомным агентом

Если конкурент и наш ритейлер оба работают на одном универсальном агентном рантайме (а они работают, потому что рантайм коммодитизирован), повторить рантайм тривиально. Библиотека навыков кодирует накопленные паттерны компании. Это центральный стратегический тезис отчёта.

Ритейлер, вкладывающийся в кастомных агентов, но пренебрегающий библиотекой навыков, вкладывается в ту часть стека, которую конкуренты получат бесплатно. Ритейлер, вкладывающийся в навыки — даже на универсальном рантайме, — строит долговечный актив, захватывающий то, что компания реально знает.

Холодный старт: с чего реально начать

Полезное упражнение: возьмите самого опытного практика команды на час, попросите рассказать три рабочих процесса, которые он выполняет регулярно, и запишите разговор. Транскрипт — почти буквально первый черновик трёх навыков. Это работа, у которой до навыков не было очевидного дома. Теперь он есть.

Сноски

1. https://codelabs.developers.google.com/getting-started-with-antigravity-skills
2. https://github.com/anthropics/skills/tree/main/skills/skill-creator
3. https://hermes-agent.nousresearch.com/docs/user-guide/features/skills
4. https://github.com/Shubhamsaboo/awesome-llm-apps/tree/main/awesome_agent_skills/self-improving-agent-skills
5. https://adk.dev/skills/
6. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
7. https://arxiv.org/abs/2602.12670 — SkillsBench, 2025
8. https://vercel.com/blog/agents-md-outperforms-skills-in-our-agent-evals
9. https://latitude.so/blog/agent-first-comparison-guide-vs-braintrust
10. https://adk.dev/evaluate/
11. https://arxiv.org/html/2411.13768v2 — Evaluation-Driven Development
12. https://arxiv.org/abs/2508.16260 — MCPVerse
13. https://research.trychroma.com/context-rot — Chroma Research, 2025
14. https://arxiv.org/abs/2406.12045 — tau-bench (Yao et al., 2024)
15. https://arxiv.org/abs/2601.06112 — ReliabilityBench
16. https://arxiv.org/abs/2601.17087 — «Lost in Simulation»
17. https://google.github.io/agents-cli/ — Google Agents CLI
18. Liu, Zhao, Shang и Shen (2026). Reverse-engineering Claude Code v2.1.88; сайт-компаньон ccunpacked.dev
19. Liu et al., «Lost in the Middle» (TACL 2024). https://arxiv.org/abs/2601.06112
20. Chroma Research, «Context Rot» (2025). https://arxiv.org/abs/2601.17087
21. https://developers.googleblog.com/en/developers-guide-to-building-adk-agents-with-skills/
22. https://github.com/anthropics/skills
23. https://github.com/anthropics/skills/blob/main/skills/skill-creator/SKILL.md
24. https://github.com/Shubhamsaboo/awesome-llm-apps/tree/main/awesome_agent_skills/self-improving-agent-skills
25. https://github.com/karpathy/autoresearch
26. https://arxiv.org/abs/2305.16291 — Voyager
27. https://github.com/philschmid/self-learning-skill

28. Google. Agents CLI: единый CLI для полного цикла разработки агентов ADK. https://google.github.io/agents-cli/. Источник: https://github.com/google/agents-cli. Поставляется с семью навыками: scaffold, build, eval, deploy, publish, observability.
29. Google Cloud Blog. (2026, апрель). Level Up Your Agents: официальный репозиторий навыков Google. Репозиторий: https://github.com/google/skills. Установка: `npx skills install github.com/google/skills`.
30. Google Developers Blog. (2026, март). Closing the knowledge gap with agent skills. Навык Gemini API поднимает Gemini 3.1 Pro с 28,2% до 96,6% на генерации SDK-кода (117 промптов). Источник: https://github.com/google-gemini/gemini-skills.
31. Anthropic. (2025–2026). anthropics/skills — публичный репозиторий навыков агентов: эталонная реализация стандарта + примеры (docx, pdf, xlsx, pptx, сборка MCP-серверов, создание навыков).
32. Stripe. (2026). stripe/ai — всё для ИИ-продуктов на Stripe: навык stripe-best-practices (выбор API, Connect, биллинг, безопасность).
33. Microsoft. (2026). microsoft/skills — навыки, MCP-серверы, кастомные агенты и Agents.md для заземления кодинг-агентов в Azure SDK и Microsoft Foundry.
34. VoltAgent. (2026). awesome-agent-skills — подборка 1000+ навыков от официальных команд и сообщества. Совместимы с Claude Code, Codex, Antigravity, Cursor, GitHub Copilot, OpenCode, Windsurf.
35. Skills Marketplace. (2026). https://skillsmp.com — независимый агрегированный маркетплейс 1,2M+ навыков из публичных GitHub-репозиториев, с семантическим поиском и индикаторами минимального качества (порог 2 звезды).
36. Osmani, A. (2026). addyosmani/agent-skills — производственные инженерные навыки для ИИ кодинг-агентов.